Đề tài 17

myls

Cài đặt lệnh ls
bằng C trên Linux



`-a` `-l` `-R` `-S`

Bốn cờ tác động lên bốn quyết định

Có $2^4 = 16$ tổ hợp, bao gồm cả chế độ mặc định.

-a



CHỌN MỤC

Thêm tên bắt đầu bằng dấu chấm

`name[0]`

-l

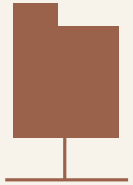


CHỌN CÁCH IN

Quyền · owner size · mtime

`struct stat`

-R

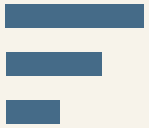


CHỌN NƠI ĐẾN

Đi vào thư mục con thật

`S_ISDIR`

-S



CHỌN THỨ TỰ

Size giảm; cùng size thì tên

`st_size`

-a thay tập kết quả, không thay cách duyệt

Cùng một thư mục: thêm -a để thấy các tên bắt đầu bằng dấu chấm.

```
$ ./myls
```

```
README.txt
database.bin
link_to_src
notes.txt
src_module
```

```
$ ./myls -a
```

```
.
..
.config_hidden
.env
README.txt
database.bin
link_to_src
notes.txt
src_module
```



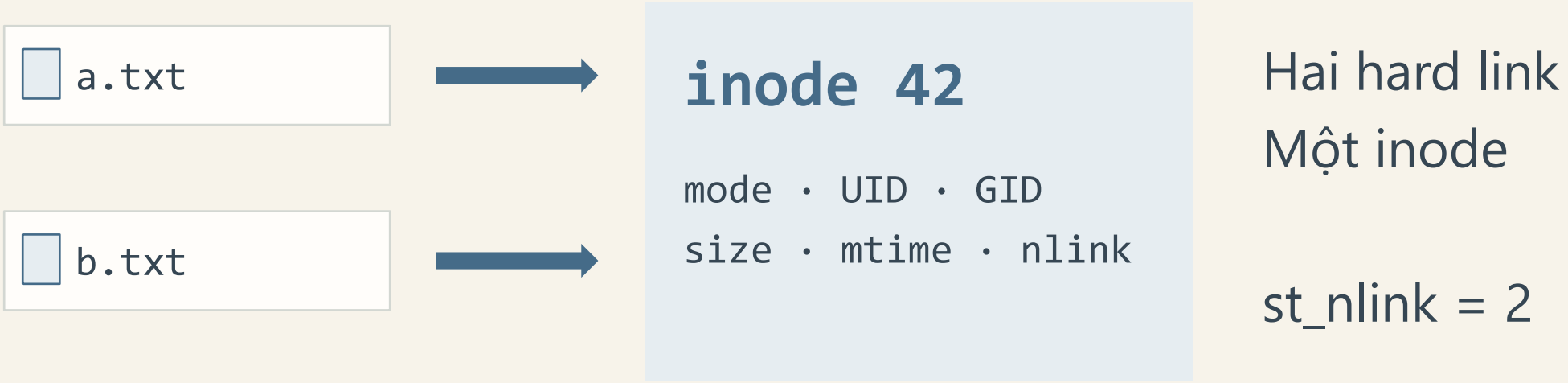
Tên ẩn bắt đầu bằng .

. và .. được in khi có -a; luôn bị loại khỏi bước đệ quy.

Tên tệp không chứa toàn bộ thông tin tệp

readdir() trả tên; lstat() lấy metadata của chính đối tượng tại đường dẫn.

MỤC TRONG THƯ MỤC



struct dirent: tên → struct stat: metadata

Một pipeline cho mọi tổ hợp cờ

Thu thập → sắp xếp → in thư mục hiện tại → duyệt tiếp nếu có -R.

1

Đọc CLI

`main + options`

→

2

Thu thập

`directory`

→

3

Sắp xếp

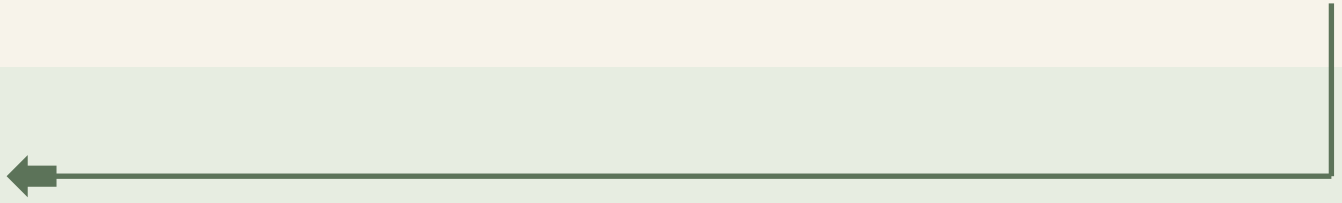
`sorting`

→

4

Hiển thị

`display`

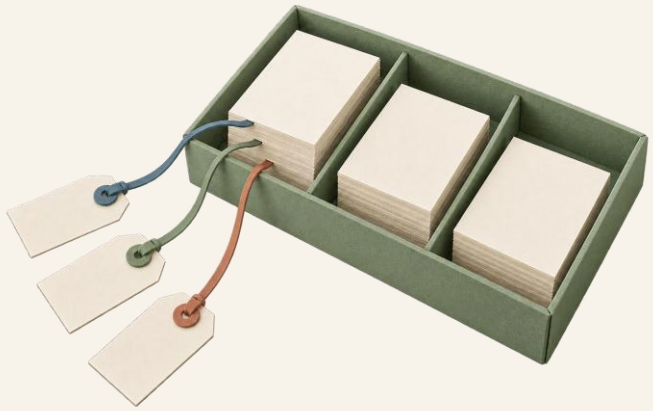


-R: lặp pipeline cho thư mục con thật

Lỗi → stderr · Exit status chỉ tăng mức nghiêm trọng: 0 → 1 → 2.

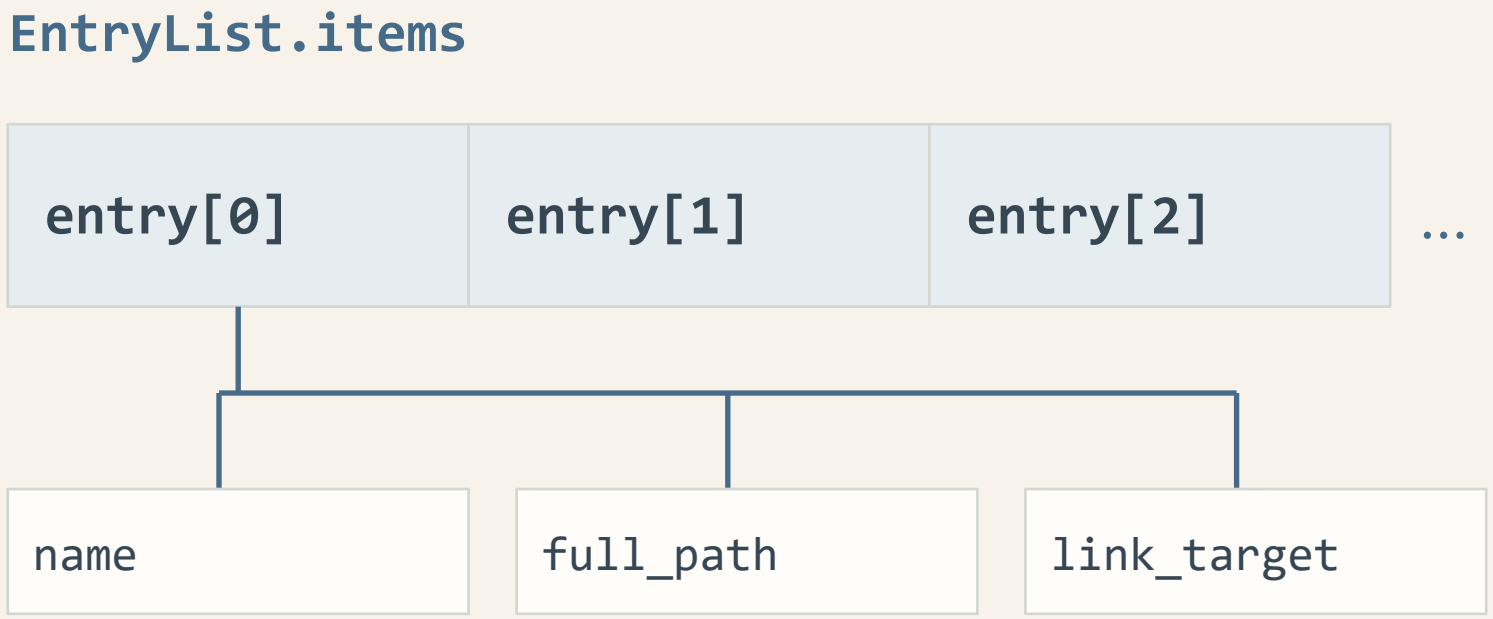
EntryList sở hữu dữ liệu của mỗi thư mục

Không giữ con trỏ d_name qua lần readdir() kế tiếp; sao chép chuỗi vào heap.



EntryList

`items · count · capacity`



Sức chứa: 64 → 128 → 256

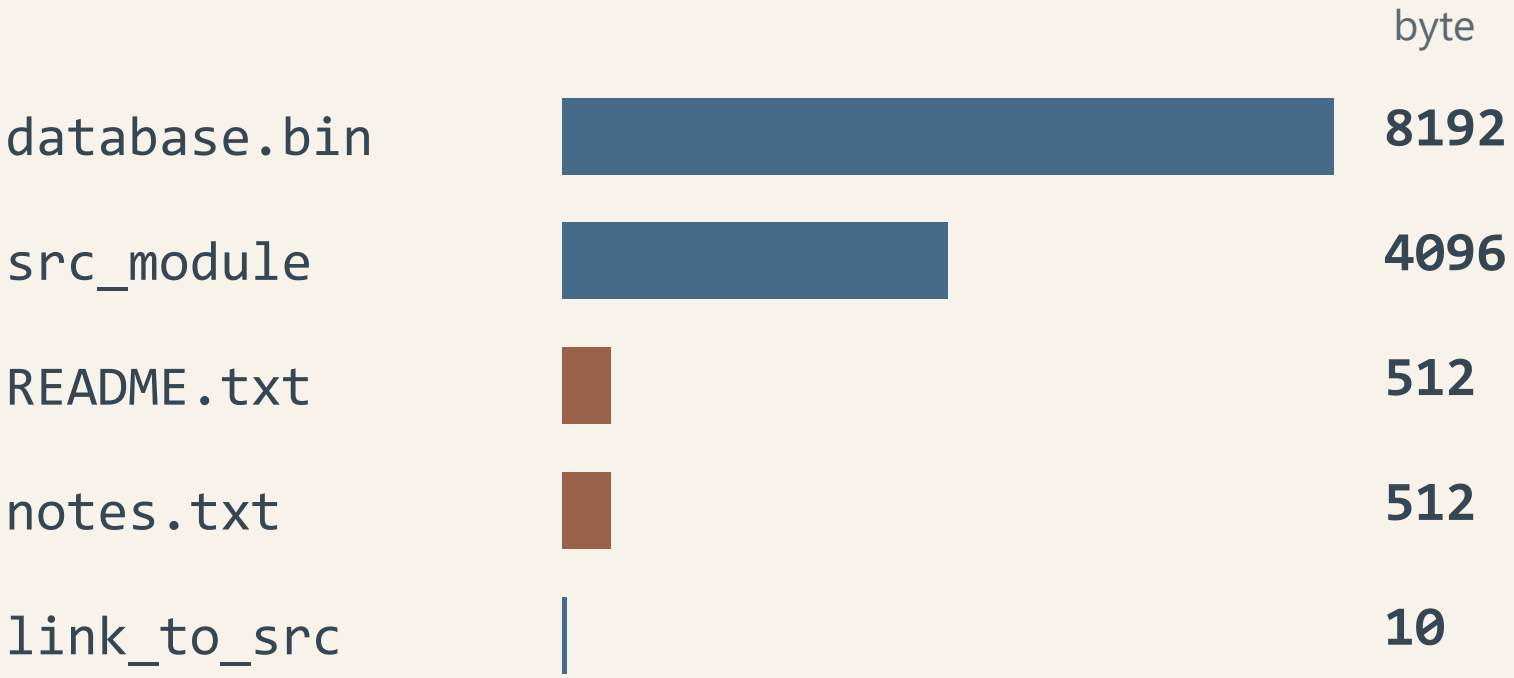
Kiểm tra tràn trước phép nhân/realloc.

entry_list_free():

Giải phóng chuỗi rồi mảng items.

-S: kích thước giảm dần, tên phá hòa

So sánh theo byte; nếu bằng nhau, so tên theo thứ tự byte.



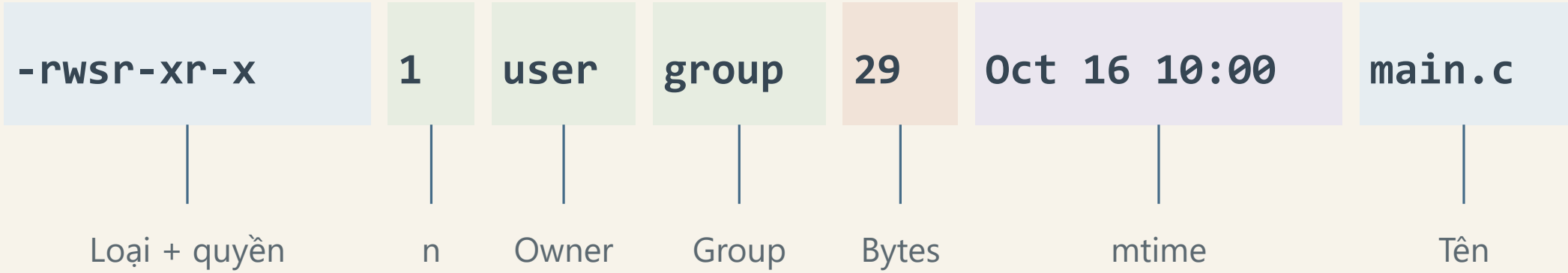


512 = 512 → README.txt trước notes.txt

So sánh > / <, tránh trừ hai kích thước gây tràn.

-l biến metadata thành một dòng có cấu trúc

Từng trường trả lời một câu hỏi: loại, quyền, sở hữu, kích thước, thời gian, tên.



Hai lượt: đo độ rộng → in canh cột.
UID/GID không có tên thì dùng số.

s = SUID + execute của owner.
Symlink: thêm "tên → đích".

lstat() nhìn link; stat() nhìn đối tượng đích

Trong cây -R: giữ metadata của link, không đi vào đích.



`stat(link_to_src)`

Metadata của
thư mục src_module/

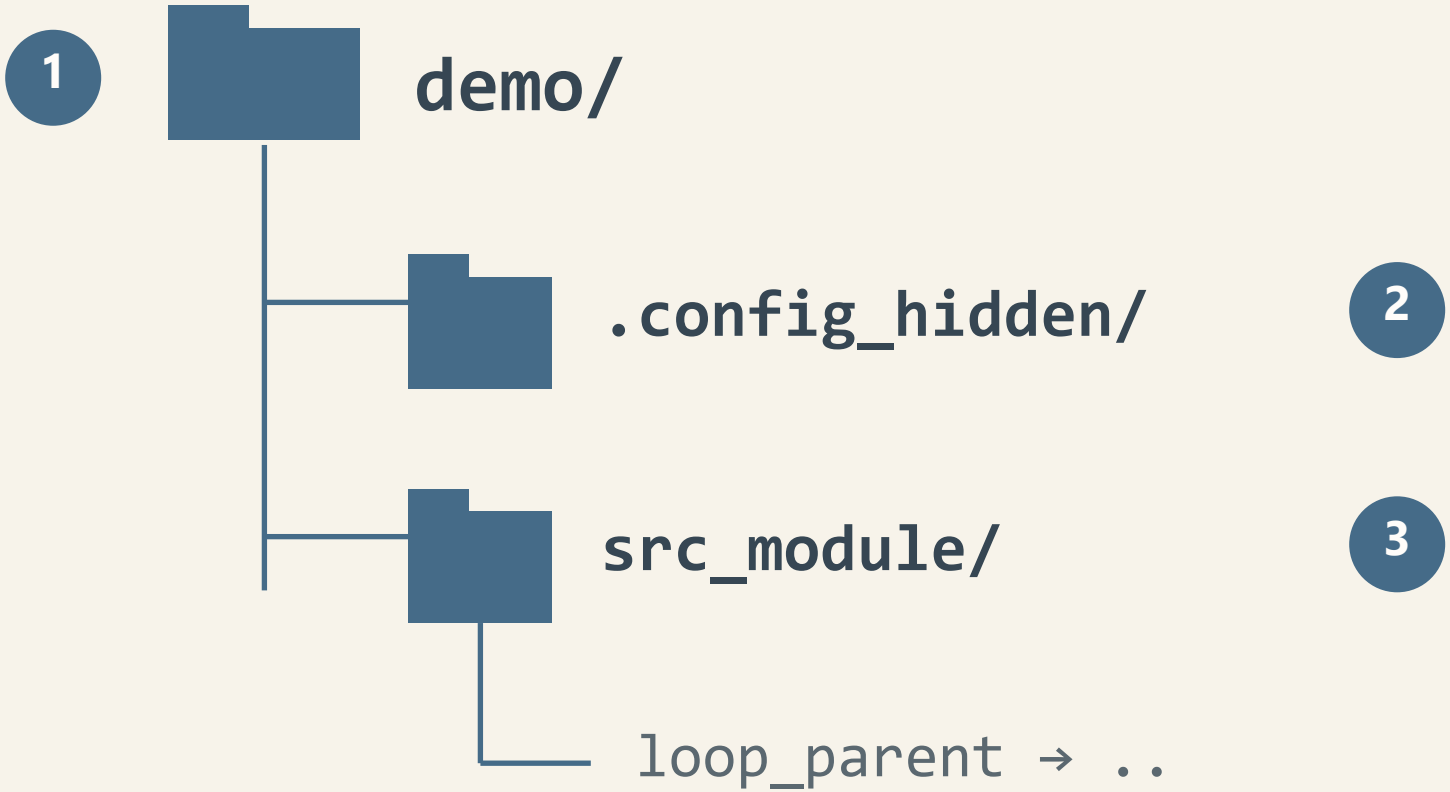
`lstat(link_to_src)`

Metadata của
chính symlink

Operand CLI trỏ tới thư mục: không -l → duyệt đích; có -l → in link.

-R đi theo cây thư mục thật

In thư mục hiện tại → duyệt con đã sắp xếp; không theo symlink.





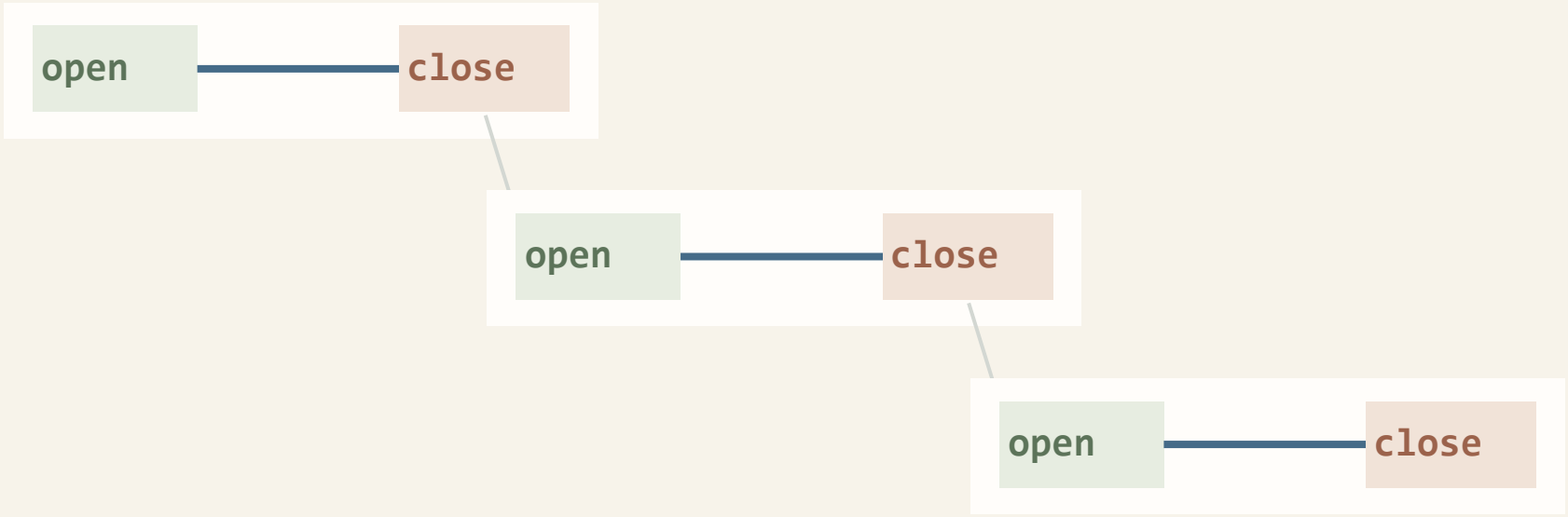
DỪNG TẠI

`.` | `..` | `symlink`

Với -aR: in, không đi vào.

Đóng DIR* trước khi gọi đệ quy

Giảm descriptor đang mở; dữ liệu cha vẫn còn trong bộ nhớ.



FD: không giữ DIR* theo mỗi độ sâu.
Stack lời gọi: vẫn tăng theo độ sâu.

RAM: tổng EntryList còn sống trên nhánh, cộng chuỗi đường dẫn.

Báo lỗi nhưng vẫn xử lý operand hợp lệ

Mã thoát giữ mức nghiêm trọng cao nhất đã gặp.

0 **Hoàn tất**

Không ghi nhận lỗi

1 **Lỗi khi duyệt**

Entry / thư mục con

2 **Lỗi nghiêm trọng**

CLI · bộ nhớ · stdout

```
$ ./myls missing_path.txt src_module
```

```
myls: cannot access 'missing_path.txt': No such file or directory
src_module:
loop_parent
main.c
```

Demo trả exit 2; lỗi ghi stderr, kết quả hợp lệ ghi stdout.

Kiểm chứng cả hành vi lẫn đường lỗi

26 ca đối chiếu GNU ls · 11 ca kiểm hợp đồng, helper và lỗi giả lập.

37 / 37

Bản thường

37 / 37

ASan + UBSan + LSan





Tổ hợp 4 cờ

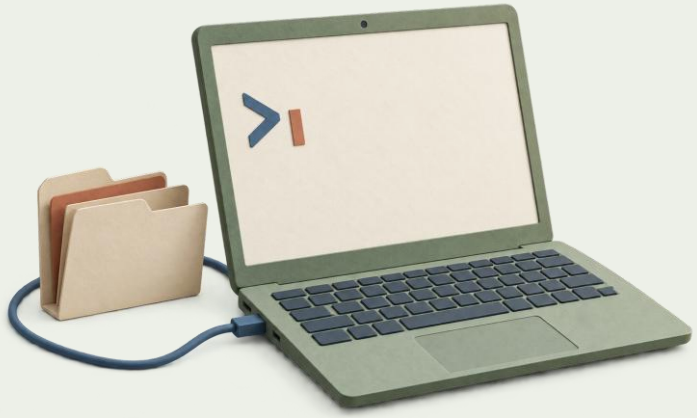
GNU: ca biên

Hợp đồng / fault / unit

Bản thường / sanitizer: FAIL 0 / 0 · SKIP 0 / 0

Demo: bốn thay đổi có thể quan sát

Mục ẩn → thứ tự → đệ quy → xử lý lỗi.



`make demo`

1. **`./myls → ./myls -a`**

   Xuất hiện mục ẩn, . và ..

2. **`./myls -lS`**

   Size giảm; hai tệp 512 byte phá hòa theo tên

3. **`./myls -alRS`**

   Vào thư mục ẩn, không theo link tới ..

4. **`./myls missing_path.txt src_module`**

   Có lỗi, vẫn in operand hợp lệ; exit 2

Phạm vi rõ giúp đánh giá kết quả đúng

Tái hiện bốn cờ trong hợp đồng đã công bố; vẫn có khác biệt với GNU ls.

HIỂN THỊ

Một tên mỗi dòng; LC_ALL=C, so tên theo byte.

Chưa có nhiều cột, màu, quoting ký tự điều khiển.

METADATA

Quyền Unix, link, device, size, time và total.

Chưa có ACL/SELinux; phụ thuộc metadata filesystem.

DUYỆT CÂY

Bỏ . / ..; không theo symlink khi đi xuống.

Chưa có tập tổ tiên (st_dev, st_ino) để chặn vòng bind mount.

Từ một lệnh quen thuộc đến cơ chế hệ thống

Đọc đúng. In đúng.
Giải phóng đúng.

HÀNH VI	4 cờ, 16 tổ hợp, cả ca lỗi
CƠ CHẾ	dirent · stat · heap · descriptor
KIỂM CHỨNG	GNU ls + sanitizer + fault injection



Trao đổi & câu hỏi

Quyền đặc biệt thay ký tự execute thế nào?

Tệp mẫu main.c được chmod 4755 trong demo.

Bit đặc biệt	Vị trí	Có x	Không có x
SUID	owner	s	S
SGID	group	s	S
Sticky	other	t	T

04755 → -rwsr-xr-x

Dùng mask S_ISUID / S_ISGID / S_ISVTX và các bit quyền.

Size, total và thời gian là ba khái niệm khác nhau

1 **`st_size`**

Byte logic của một đối tượng

2 **`st_blocks`**

Số khối 512 byte đã cấp phát

3 **`st_mtime`**

Lần sửa nội dung gần nhất

```
total (1 KiB) = (Σ st_blocks + 1) / 2
```

Gần hiện tại: HH:MM; quá nửa năm hoặc ở tương lai: năm. Test cố định TZ=UTC0.

Symlink: phân biệt CLI với mục khi duyệt

Ngữ cảnh	Không -l	Có -l
Link → thư mục, operand CLI	Liệt kê nội dung đích	In chính link
Link đứt, operand CLI	In tên link	In link → chuỗi đích
Link gặp trong thư mục -R	Chỉ in tên; không đệ quy	In link; không đi xuống

-R không biến symlink bên trong cây thành cạnh đệ quy.

Fault injection làm đường lỗi có thể tái lập

LD_PRELOAD ép một số lời gọi thất bại theo điều kiện; kiểm cách xử lý của chương trình.

`readdir / closedir`	→	Báo lỗi; giữ mức exit phù hợp
`readlink / realloc`	→	Dọn dữ liệu dở; không crash
`/dev/full (stdout)`	→	Phát hiện lỗi ghi và flush; exit 2

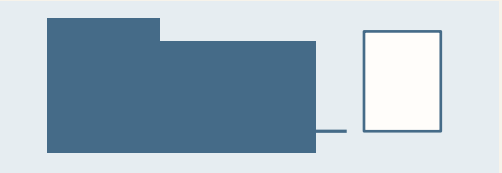
Helper nội bộ: bit quyền, device, total, overflow; cũng được build với sanitizer.

Chạy trên WSL: chọn đúng nơi tạo fixture



/tmp · Linux filesystem

Fixture tự động ở đây.
chmod, symlink và special bits
được thử trên môi trường Linux.



/mnt/p · ổ Windows

Có thể lưu và build repo.
Quyền phụ thuộc DrvFs,
mount options và metadata.

Không kết luận “WSL luôn hiện 0777”; kiểm mount và cấu hình máy đang chạy.

Đọc tiếp và tái lập từ đúng nguồn

README để chạy; tài liệu kỹ thuật để học; log để kiểm chứng.

01 **Chạy chương trình** — `README.md`

Build · test · sanitizer · demo

02 **Hiểu triển khai** — `docs/TECHNICAL_DEEP_DIVE.md`

Khái niệm → trace → cờ → tài nguyên → lỗi

03 **Xem minh chứng** — `evidence/test_results.json`

Môi trường · source hash · từng ca · demo

Tham chiếu: GNU Coreutils (ls) · Linux man-pages (stat, readdir, readlink) · Microsoft Learn (WSL).